Pre Study

Literature review

Bruno Carchia
carchia@kth.se

April 28, 2026

1 Introduction

This section provides a critical analysis of the literature and technical reports that informed the selection of the methodologies and technologies employed in this research. Each reviewed paper contributed specifically to the architectural design or the strategic formulation of the Reinforcement Learning agent. To ensure clarity, each work is presented using the following structure:

- **Core Summary:** A brief overview of the research objectives and findings of the paper.
- **Technical Contribution:** An explanation of how the paper's insights directly influenced the development of this thesis.

2 An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line

This study [2] addresses the Rolling Stock Circulation (RSC) problem by formulating it as a multi-objective optimization task within a Deep Reinforcement Learning framework. The primary objectives identified are the minimization of fleet utilization, the balancing of maintenance workloads, and the stabilization of depot stock levels.

State Space Representation

The authors define a hierarchical state representation where the global state s^t is a concatenation of individual rolling stock statuses s_k^t. The local state for each asset k is defined as a six-dimensional feature vector:

$$s_k^t = \left(n_k^t, \bar{c}_k^t, \bar{b}_k^t, f_k^t, o_k^t, g_k^t\right)$$

This vector captures critical operational constraints, including cumulative trip counts (n_k^t), remaining service life ($\bar{c}_k^t$), and turnaround readiness (f_k^t). Notably, the inclusion

of operation direction (o_k^t) and a timetable pointer (g_k^t) ensures the agent maintains the Markov property by encoding the physical feasibility of future assignments.

Action Space

The action space $A = \{1, 2, \ldots, |K|\}$ is discrete, where each action corresponds to the selection of a specific physical asset from the fleet. To handle the "hard constraints" of rail operations (e.g., assigning a busy train), the authors utilize a penalty-based approach rather than strict action masking. This forces the agent to learn the underlying feasibility logic through the negative reinforcement of invalid selections.

Reward Mechanism

A dual-reward structure is employed to handle the credit assignment problem:

1. **Dense Step-wise Reward (r'):** Focused on immediate utilization efficiency and constraint satisfaction. It penalizes unnecessary depot extractions (r_a) and rewards the selection of idle assets with high turnaround times (r_v), promoting a "First-In, First-Out" logic.

2. **Sparse Terminal Reward (Z):** A weighted sum of normalized objectives calculated at the end of the operation horizon. This term targets fleet-wide fairness (workload standard deviation z_3) and depot equilibrium (z_2).

(a) Actor Network

(b) Critic Network

Figure 1: Architecture of the proposed Actor-Critic Neural Network. Taken from (Yang et al., 2025 [2])

Neural Network Architecture

The problem is solved using the Proximal Policy Optimization (PPO) algorithm. As illustrated in Figure 1, the Actor and Critic networks utilize a fully-connected Multi-Layer Perceptron (MLP) structure with hyperbolic tangent (tanh) activation functions.

Technical Contribution and Relevance to Thesis

This paper provides the foundational logic for my research, particularly in the structuring of sparse and dense rewards to balance local efficiency with global maintenance goals. However, while the authors utilize a global MLP that scales linearly with fleet size, my work improves upon this by implementing a **Shared-Weight MLP**. This ensures that the model is permutation-invariant and can generalize to varying fleet sizes without re-training—a significant limitation observed in the original paper's architecture.

Feature	Reference Paper	This Thesis
Algorithm	PPO (Actor-Critic)	PPO (Actor-Critic)
Action Space	Discrete (Fixed $\|K\|$)	Continuous Weights (Scalable)
NN Architecture	Global Flat MLP	Shared-Weight Architecture
Observation Space	Fixed Global Vector	Masked Relational Features
Maintenance Logic	Reactive (Fixed Thresholds)	Proactive (Weibull-based)
Simulation Type	Step-based	Event-driven

Table 1: Comparative analysis between the reference study and the proposed DRL approach.

3 Deep Reinforcement Learning for Integrated Train Scheduling and Rolling Stock Circulation

A significant contribution to integrated rail optimization is presented by Zhao et al. [3], who propose a Deep Reinforcement Learning (DRL) approach to solve the joint problem of train scheduling and rolling stock circulation. The authors utilize a Hybrid Proximal Policy Optimization (HPPO) algorithm to manage a hybrid action space characterized by both continuous scheduling decisions and discrete rolling stock assignments. The primary objective is to minimize a multi-objective cost function comprising passenger waiting time and operational expenses under complex constraints.

In this framework, the DRL agent acts as a dispatcher, sequentially determining section running times, station dwell times, and the allocation of rolling stock units—either by deploying units from the depot or reassigning units arriving from previous services.

State Space Representation

The state space S_n at step n captures the temporal and logistical dynamics of the network:

$$S_n = (d_{n,i}, a_{n,i}, l_{n,i}, o_{n,k}) \tag{1}$$

where $d_{n,i}$ and $a_{n,i}$ denote departure and arrival times at station i, $l_{n,i}$ represents the passenger queue length post-departure, and $o_{n,k}$ indicates the availability time of rolling stock unit k.

Action Space Representation

The action space A_n is hybrid, formulated as:

$$A_n = (h_{n+1}, r_{n+1,i}, e_{n+1,i}, f_{n+1,k}) \tag{2}$$

Here, the headway (h_{n+1}), section running times ($r_{n+1,i}$), and dwell times ($e_{n+1,i}$) are continuous variables, while the rolling stock assignment ($f_{n+1,k} \in \{0, 1\}$) is a discrete binary decision.

Reward Mechanism

The reward function is defined as the negative weighted sum of three components:

1. *Passenger Waiting Costs*: Calculated as the sum of waiting time for newly arrived passengers and those stranded due to capacity limits from previous services.

2. *Operational Costs*: Defined by the total cycle time for service n, calculated as $(a_{n,2I} - d_{n,I})$.

3. *Usage Costs*: Proportional to the total number of active rolling stock units $\sum_{k \in K} \eta_k$.

Neural Network Architecture

To handle the hybrid nature of the actions, the architecture employs a dual-actor system 2. A discrete actor (π_{θ_d}) outputs the probability distribution for rolling stock assignments, while a continuous actor (π_{θ_c}) parameters a multivariate Gaussian distribution to sample scheduling variables.

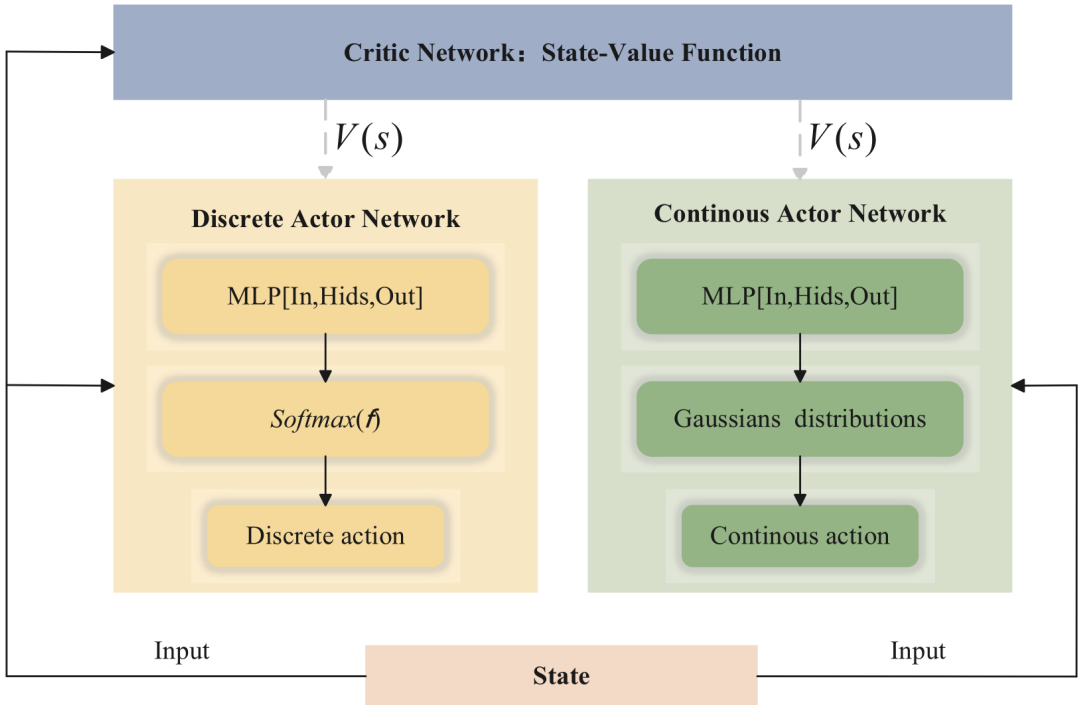

Figure 2: Architecture of the proposed Actor-Critic Neural Network. Taken from (Zhao et al., 2026 [3])

Relevance and Comparative Analysis

While the study by Zhao et al [3]. addresses a more geographically complex passenger network, it provides a vital methodological benchmark for this thesis. Both approaches utilize action masking to enforce operational constraints; however, several key distinctions exist. While Zhao et al. [3] optimize for passenger flows using a hybrid HPPO actor-critic model, this thesis focuses on freight logistics (specifically the transport of timber—within a two-line system).

Furthermore, instead of the dual-actor hybrid approach (HPPO) used in the cited study, this research employs a standard PPO algorithm with a single continuous actor. In this thesis, discrete requirements are handled by mapping continuous outputs to discrete values via a specialized mapping strategy, rather than maintaining a separate discrete actor network.

4 Solving high-speed rolling stock planning with an improved LSTM-PPO reinforcement learning algorithm

The work presented by Su et al. [1] investigates the Rolling Stock Planning (RSP) problem utilizing a PPO framework integrated with a Long Short-Term Memory (LSTM) network. To prevent the agent from converging on local optima, the authors incorporate a simulated annealing strategy, effectively balancing global exploration with local refinement. The choice of an LSTM architecture is specifically intended to capture temporal dependencies and provide the agent with a recursive memory of the episode's history.

A fundamental assumption of this study is the prioritization of a fixed maintenance threshold over train departures, ensuring safety constraints are satisfied. The operational environment is modeled with three stations and a fixed scheduling regime, establishing a scenario closely aligned with the parameters of this thesis.

State Space Representation

The state at time step t is defined by the history of assignments V_t and the current task x_t. The history V_t represents a comprehensive record of all train services already allocated to the v rolling stock units (RSUs), while x_t denotes the specific service currently awaiting assignment:

$$V_t = \left\{U_t^1, U_t^2, \ldots, U_t^v\right\} = \left\{\left\{u_1^1, u_2^1, \ldots, u_r^1\right\}, \ldots, \left\{u_1^v, u_2^v, \ldots, u_r^v\right\}\right\}$$

Action Space Representation

The agent is tasked with mapping the pending train service (x_t) to an available RSU (v_i). The resulting action is formulated as:

$$a_t = (v_i, x_t)$$

Consistent with the methodology adopted in this research, the action space is constrained by a filtering mechanism that automatically excludes actions violating operational requirements, such as maintenance schedules.

Reward Mechanism

The reward function is a multi-objective scalar derived from three primary components, each modulated by specific hyperparameters:

1. r_t^{connect}: A positive reward for establishing a feasible connection between successive train services on the same RSU.

2. r_t^{save}: An incentive aimed at minimizing the total number of RSUs required for operations.

3. r_t^{penalty}: A penalty term for the violation of hard constraints, primarily regarding fixed maintenance intervals.

Relevance and Comparative Analysis

The problem addressed by Su et al. [1] serves as a foundational benchmark for this thesis, although it represents a slightly simplified version of the real-world system as it focuses exclusively on RSU efficiency without considering passenger demand.

There is a significant alignment between this thesis and Su et al. [1] regarding the prioritization of system safety; both models enforce maintenance requirements as a higher priority than schedule adherence. However, the maintenance modeling differs in flexibility: while Su et al. [1] restricts maintenance to nighttime windows, this thesis allows the agent to execute early maintenance at any time to accommodate operational needs. Furthermore, while the present work omits dwell times as a deliberate simplification, this feature remains a viable candidate for future integration. Finally, although this thesis utilizes a Multi-Layer Perceptron (MLP) architecture rather than the LSTM employed by Su et al. [1], the algorithmic structure of their work provided significant inspiration for the development of our proposed solution.

Algorithm 1 LSTM-PPO for rolling stock planning.

Require: Initialized actor π_θ, critic V_ϕ, LSTM state h_0, learning rates $\alpha_\theta, \alpha_\phi$, clipping parameter ϵ, epochs K

Initialize rolling stock planning environment
for each training iteration **do**
  Collect episodes:
  for each circulation planning episode **do**
    Reset environment and initialize LSTM hidden state h_0
    for each train service step t **do**
      Observe current state s_t (e.g. RSU status, depot availability, task x_t)
      Compute action distribution $\pi_\theta(a_t \mid s_t, h_{t-1})$
      Sample assignment decision a_t
      Execute a_t, observe reward r_t and next state s_{t+1}
      Store (s_t, a_t, r_t, h_{t-1}) in buffer
      Update hidden state: $h_t \leftarrow \text{LSTM}(s_t, h_{t-1})$
    end for
  end for
  Estimate advantages $\hat{A}_t$ and returns $\hat{R}_t$ via GAE
  for K epochs **do**
    Sample mini-batches from buffer
    for each mini-batch **do**
      Compute probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t, h_{t-1})}{\pi_{\theta_{\text{old}}}(a_t \mid s_t, h_{t-1})}$$

      Compute clipped surrogate loss:

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

      Update actor parameters: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta L^{\text{PPO}}$
      Update critic loss: $L_V = \|V_\phi(s_t, h_{t-1}) - \hat{R}_t\|^2$
      Update critic parameters: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi L_V$
    end for
  end for
end for

Figure 3: PPO-LSTM Algorithm. Taken from (SU et al., 2026 [1])

References

[1] Xiangyu Su, Yixiang Yue, Bin Guo, Zanyang Cui, and Boyan Li. Solving high-speed rolling stock planning with an improved lstm-ppo reinforcement learning algorithm. *Transportmetrica A Transport Science*, 2026.

[2] Yuhua Yang, Haoyang Huo, Nikola Bešinović, Yichen Sun, and Shaoquan Ni. An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line. *Transportation Research Part C: Emerging Technologies*, 171:104976, 2025.

[3] Xiaoli Zhao, Dewei Li, and Xinyu Bao. A deep reinforcement learning approach for integrated optimization of train scheduling and rolling stock circulation planning. *Computers & Industrial Engineering*, 213:111784, 2026.